

Stati dell'albero di ricerca

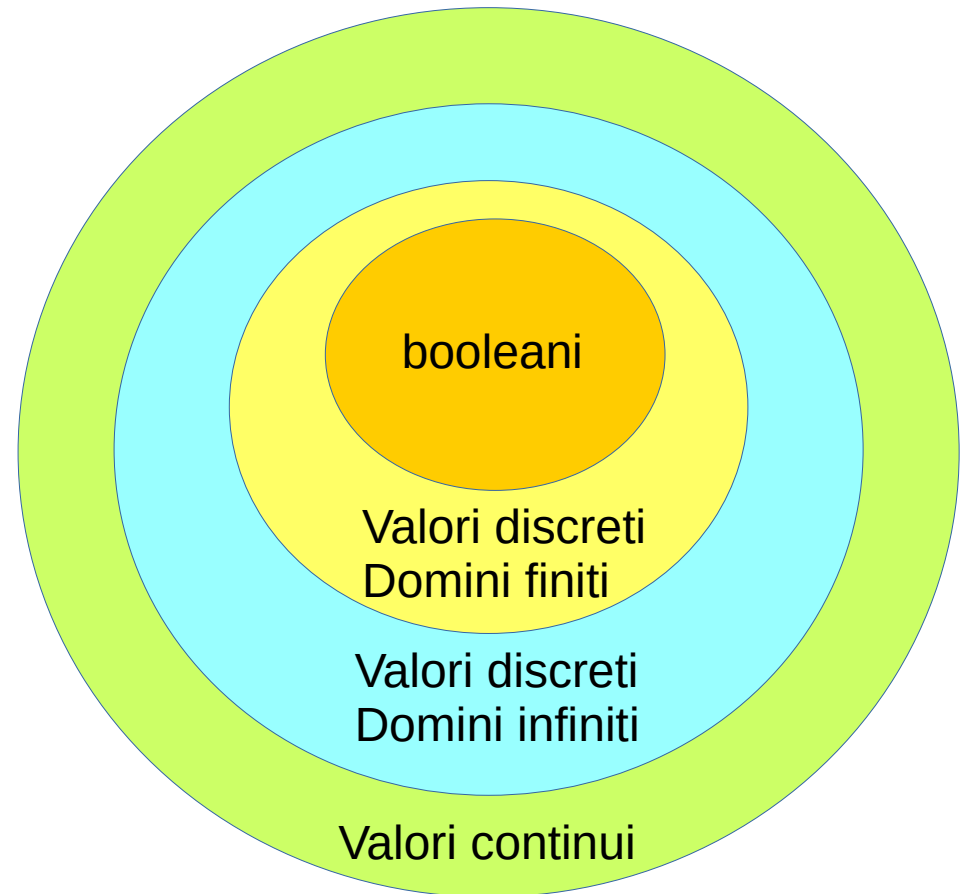
- Uno **stato** è un assegnamento di valori:
 $\{X_1 = v_1, X_2 = v_2, \dots, X_n = v_n\}$
- L'**ordine** (il **cammino**) con cui i valori vengono assegnati alle variabili è irrilevante sul risultato
- È possibile applicare metodi di ricerca già visti (esempio: blind)

Tipi di domini

- **Domini finiti:**
è possibile enumerare i vincoli mettendo in relazione i diversi valori (es. Australia)
- **Domini infiniti:**
non è possibile enumerare i vincoli,
si usano **linguaggi di specifica**, esempio per dire che Lavoro2 deve iniziare almeno 5 giorni dopo Lavoro1 si potrebbe scrivere: $\text{Lavoro1} + 5 < \text{Lavoro2}$
- **Domini continui**
la programmazione lineare permette di risolvere CSP in cui i **vincoli sono disuguaglianze lineari** che specificano una regione convessa

Variabili

- Possono essere:
 - A valori discreti
 - Booleani (NP-completa)
 - A domini finiti
 - A domini infiniti
 - A valori continui



Arità dei vincoli

- **Vincoli unari**

coinvolgono una variabile e un valore, esempio $T \neq G$ il colore della Tasmania deve essere diverso da verde

- **Vincoli binari**

coinvolgono due variabili e possono essere rappresentati come archi di un grafo, esempio $WA \neq NT$

- **Vincoli a tre o più variabili**

coinvolgono un numero qualsiasi di variabili, possono essere rappresentati da ipergraf (grafi con archi che connettono più di due nodi), esempio $diverse(WA, NT, SA)$. A volte possono essere scomposti in vincoli binari

Vincoli a 3 e più variabili, esempio

Vincoli a 3 e più variabili, esempio

- **Criptoaritmetica**: gioco in cui a ogni lettera corrisponde una cifra diversa, bisogna trovare la sostituzione corretta

$$\begin{array}{r} \text{S E N D} \\ + \text{M O R E} \\ \hline \text{M O N E Y} \end{array}$$

Vincoli a 3 e più variabili, esempio

- **Criptoaritmetica**: gioco in cui a ogni lettera corrisponde una cifra diversa, bisogna trovare la sostituzione corretta

$$\begin{array}{r} \text{S E N D} \\ + \text{M O R E} \\ \hline \text{M O N E Y} \end{array}$$

Dominio di S e M: $[1, 2, 3, 4, 5, 6, 7, 8, 9]$

Dominio di E, N, D, O, R, Y:

$[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]$

Vincoli: a lettere diverse valori diversi

Vincoli a 3 e più variabili, esempio

- **Criptoaritmetica**: gioco in cui a ogni lettera corrisponde una cifra diversa, bisogna trovare la sostituzione corretta

$$\begin{array}{r} \text{S E N D} \\ + \text{M O R E} \\ \hline \text{M O N E Y} \end{array}$$

questo vincolo coinvolge
tutte le variabili (non è
binario):

$$(\textcolor{red}{1000}*\textcolor{red}{S}+\textcolor{red}{100}*\textcolor{red}{E}+\textcolor{red}{10}*\textcolor{red}{N}+\textcolor{red}{D}+\textcolor{blue}{1000}*\textcolor{blue}{M}+\textcolor{blue}{100}*\textcolor{blue}{O}+\textcolor{blue}{10}*\textcolor{blue}{R}+\textcolor{blue}{E}) = (\textcolor{brown}{10000}*\textcolor{brown}{M}+\textcolor{brown}{1000}*\textcolor{brown}{O}+\textcolor{brown}{100}*\textcolor{brown}{N}+\textcolor{brown}{10}*\textcolor{brown}{E}+\textcolor{brown}{Y})$$

Vincoli e criteri di preferenza

- I vincoli possono essere più o meno rigidi
- Distinguiamo fra **vincoli** veri e propri e **criteri di preferenza**
- Vincoli e criteri di preferenza si differenziano sul modo in cui la loro violazione impatta sulla soluzione:
 - Una soluzione **deve soddisfare tutti i vincoli**
 - Una soluzione **può violare uno o più criteri di preferenza**
 - Il soddisfacimento dei criteri di preferenza permette di ordinare le soluzioni identificando quelle preferibili e quelle meno preferibili

Esempio, criterio di preferenza

- Nella definizione dell'orario di lezione un docente può dire di preferire le lezioni del mattino
- Questo "vincolo" va composto con quelli degli altri docenti
- Risultato: l'orario prevede che il docente faccia lezione sempre al pomeriggio 😞

11-12	(Aula F)	Sist. AS, T2 (Laboratorio Dijkstra)	(Aula F)	(Aula A)	(Laboratorio Turing)
12-13	Sist. (Aula F)	Sist. AS, T2 (Laboratorio Von Neumann) Sist. AS, T2 (Laboratorio Dijkstra)	Sist. (Aula F)	Sist. AS (Aula A)	Prog. II Lab (Laboratorio Turing)
13-14					
14-15	Sist. Int (Aula F)	Logicatut (Aula C) Sist. (Aula F)	Sist. AS (Aula C)	Logicatut (Aula B) Sist. Int (Aula F)	Logicatut (Aula F)
15-16	Sist. Int (Aula F)	Logicatut (Aula C) Sist. (Aula F)	Sist. AS (Aula C)	Logicatut (Aula B) Sist. Int (Aula F)	Logicatut (Aula F)
16-17	Sist. AS, T2 (Laboratorio Von Neumann)				
17-18	Sist. AS, T2 (Laboratorio Von Neumann)				
18-19					

La soluzione non è ottimale, perché non soddisfa la preferenza, ma potrebbe essere la migliore fra quelle possibili

Rappresentare gli stati

- Stato = assegnamento
- Modi diversi di rappresentare lo stato influiscono sul tempo della ricerca
- Approccio di forza bruta: generate-and-test
Usa solo informazioni di stato
- Ricerca blind con backtracking
Usa anche i vincoli

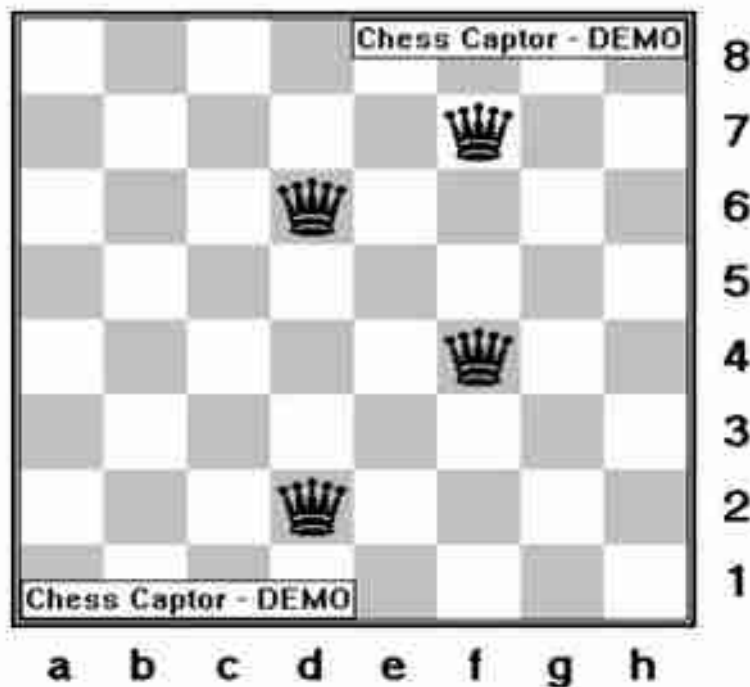
Usare solo informazioni di stato è pratico?

Generate-and-test

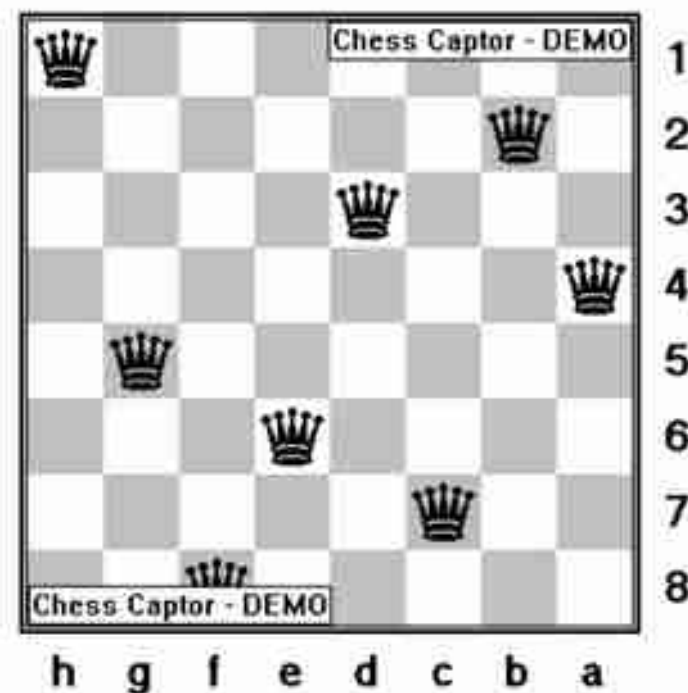
- Si può adottare una **rappresentazione a stato completo**, cioè utilizzare solo **assegnamenti completi**, che possono o meno essere consistenti
- **Generate-and-test**, metodo per la risoluzione di CSP molto semplice (ma costoso):
 - Finché non hai una soluzione e ci sono delle alternative:
 - 1) Genera un assegnamento completo
 - 2) Controlla se è consistente
 - 3) Se sì è una soluzione, esci dal ciclo
 - 4) Se no torna al passo 1
 - Se hai una soluzione restituiscila
 - Altrimenti fallimento
- Può richiedere l'esplorazione dell'intero spazio degli assegnamenti!

Esempio: problema delle 8 regine

- Posizionare 8 regine su una scacchiera 8x8 in modo tale che nessuna risulti sotto attacco di un'altra
- Il problema ha 92 soluzioni distinte (12 se si considerano solo quelle non ottenibili da altre per simmetria)



NO



SOLUZIONE



Generate-and-test: 8 regine

Q			
Q			
Q			
Q			

	Q		
Q			
Q			
Q			

		Q	
Q			
Q			
Q			

			Q
Q			
Q			
Q			

Q			
	Q		
Q			
Q			

Q			
		Q	
Q			
Q			

■ ■ ■

NB: le scacchiere sono 4x4 per rendere l'idea del procedimento senza occupare troppo spazio

Quanto tempo ci vorrà?

La risposta dipende da come abbiamo rappresentato il problema

8 regine come CSP

- **Primo modo (esempio):**
 - **Variabili:** $Q_1, Q_2, \dots, Q_8$
ognuna rappresenta la posizione di una regina
 - **Domini:** le posizioni sono numerate in modo crescente
 $1, 2, 3, \dots, 16, 17, 18, \dots, 64$
 - La soluzione riportata prima corrisponde all'assegnamento:
 $Q_1 = 1, Q_2 = 15, Q_3 = 21, Q_4 = 32, Q_5 = 34, Q_6 = 44, Q_7 = 54, Q_8 = 59$
 - **Problema:**
 - se ci sono **n variabili** ognuna delle quali può assumere **d valori**, si hanno **d^n possibili assegnamenti**
 - Nell'esempio ben oltre 28.100.000.000.000 possibili assegnamenti

8 regine come CSP, versione 2

- **Secondo modo:**
ogni regina deve per forza occupare una diversa colonna quindi **assegnamo ciascuna regina alla colonna di numero uguale** (esempio: Q1 sta nella colonna 1)
- **Variabili:** Q1, ..., Q8
- **Dominio:** 1, ... 8
il numero di riga che completa le coordinate della regina
- La soluzione di prima sarà quindi rappresentata come:
Q1 = 1, Q2 = 5, Q3 = 8, Q4 = 6, Q5 = 3, Q6 = 7, Q7 = 2, Q8 = 4
- **Va molto meglio:** abbiamo “solo” $8^8 = 16.777.216$ assegnamenti alternativi ma sono ancora tanti ...



Generate-and-test: 8 regine

- E se avessimo un numero maggiore di regine?
- In generale per N regine si hanno N^N configurazioni
- Se avessimo 16 regine le configurazioni sarebbero 18.446.744.073.709.551.616
- ~ 1200 anni di computazione

2) Usiamo più conoscenza: introduciamo i vincoli

Per passi, a partire da una semplice ricerca in profondità con backtracking, vedremo come si può rendere più efficiente la ricerca usando più e meglio l'informazione a disposizione

Generate-and-test: 8 regine

- Fino ad ora non abbiamo rappresentato i **vincoli**:
 - Date due qualsiasi regine Q_i e Q_j queste devono essere posizionate in modo che nessuna delle due sia attaccata dall'altra
 - Due regine si attaccano quando:
 - Occupano la stessa colonna:
impossibile nella nostra rappresentazione
 - Occupano la stessa riga:
aggiungiamo il vincolo $Q_i \neq Q_j$
 - Occupano la stessa diagonale:
aggiungiamo il vincolo $|i - j| \neq |Q_i - Q_j|$

Ricerca di una soluzione in profondità

- Esploriamo lo spazio degli stati (dei possibili assegnamenti) utilizzando una ricerca depth-first con backtracking
- Ricerca non informata + vincoli
per decidere quando potare un cammino
- La limitatezza dei cammini rende ragionevole la scelta della ricerca in profondità, tuttavia spesso branching factor elevato

Ricerca di una soluzione con backtracking

- Siano:
 - **n** = numero delle variabili
 - **d** = numero medio dei valori possibili per ciascuna variabile
 - Uno qualsiasi dei valori può essere assegnato a una qualsiasi delle variabili
- Il branching factor sarà **n*d** al primo livello, **(n-1)*d** al secondo (perché una variabile è stata fissata), eccetera
 - l'albero avrà **n! * dⁿ** foglie

Esempio Australia

7 variabili

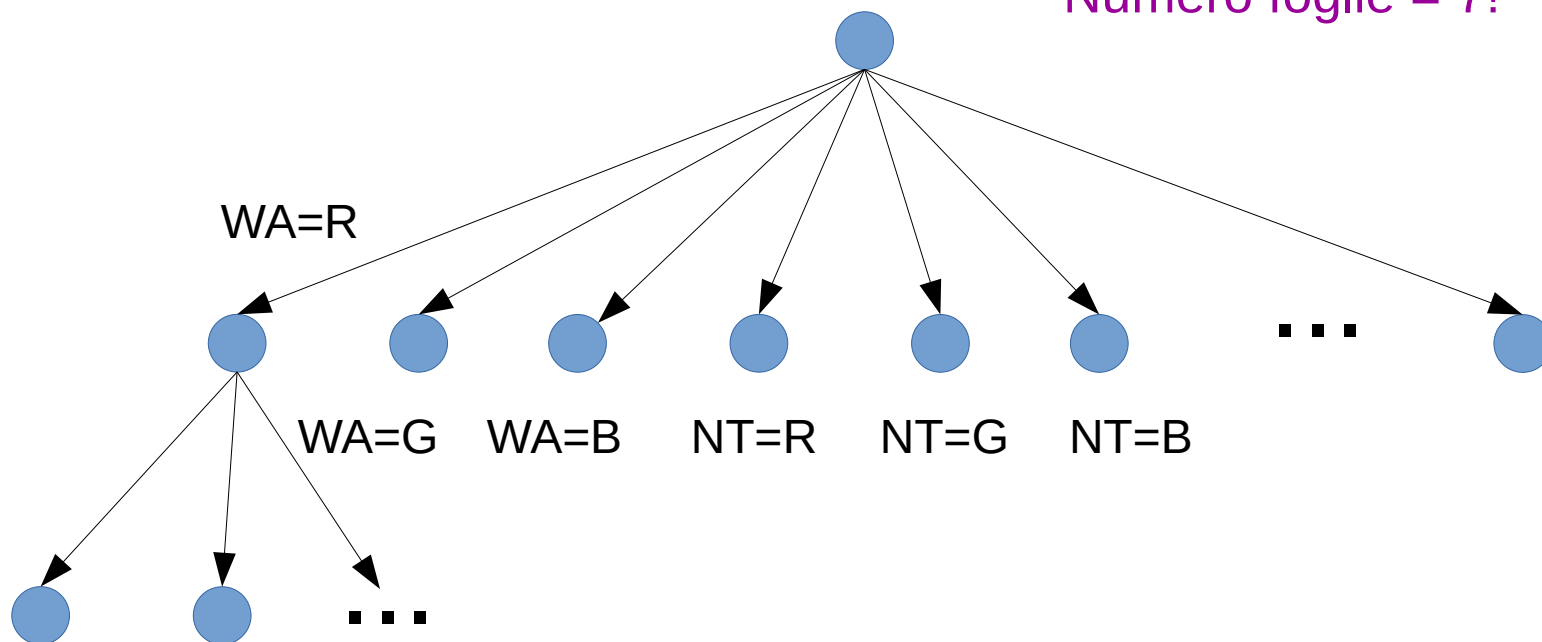
3 valori per variabile

Branching factor liv. 1 = 21

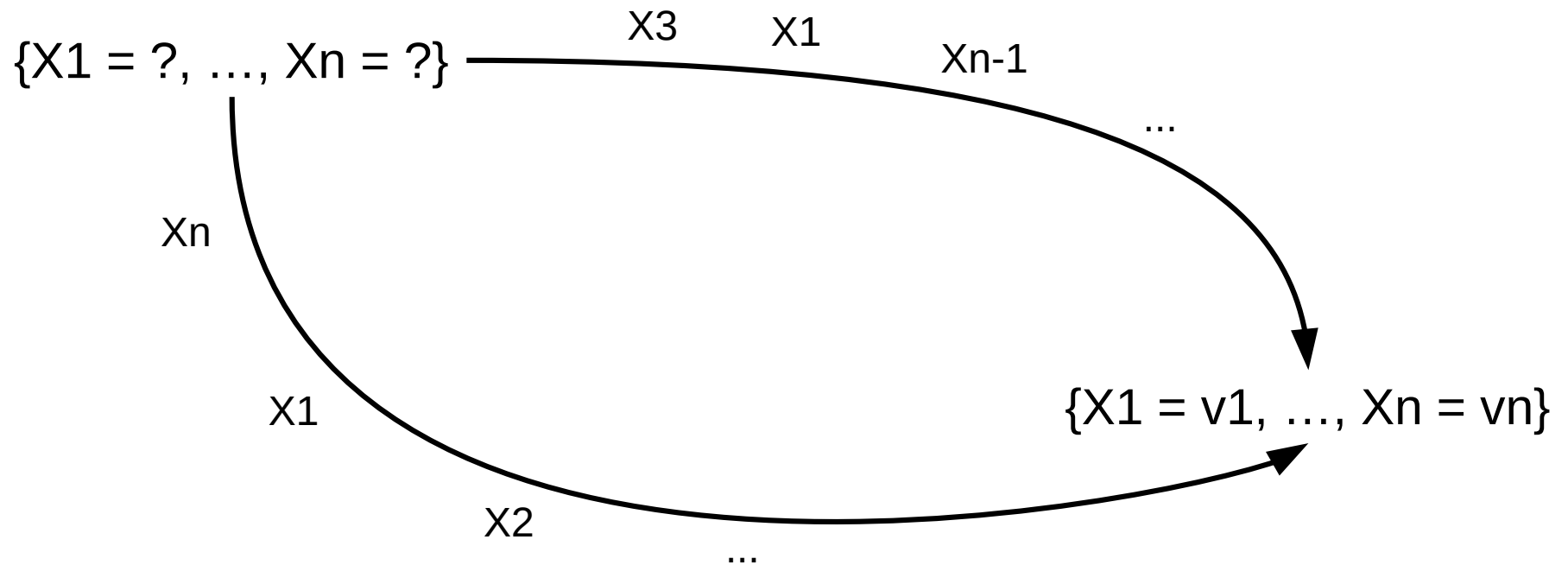
Branching factor liv. 2 = 18

...

Numero foglie = $7! * 3^7 = 11.022.480$



I CSP sono commutativi

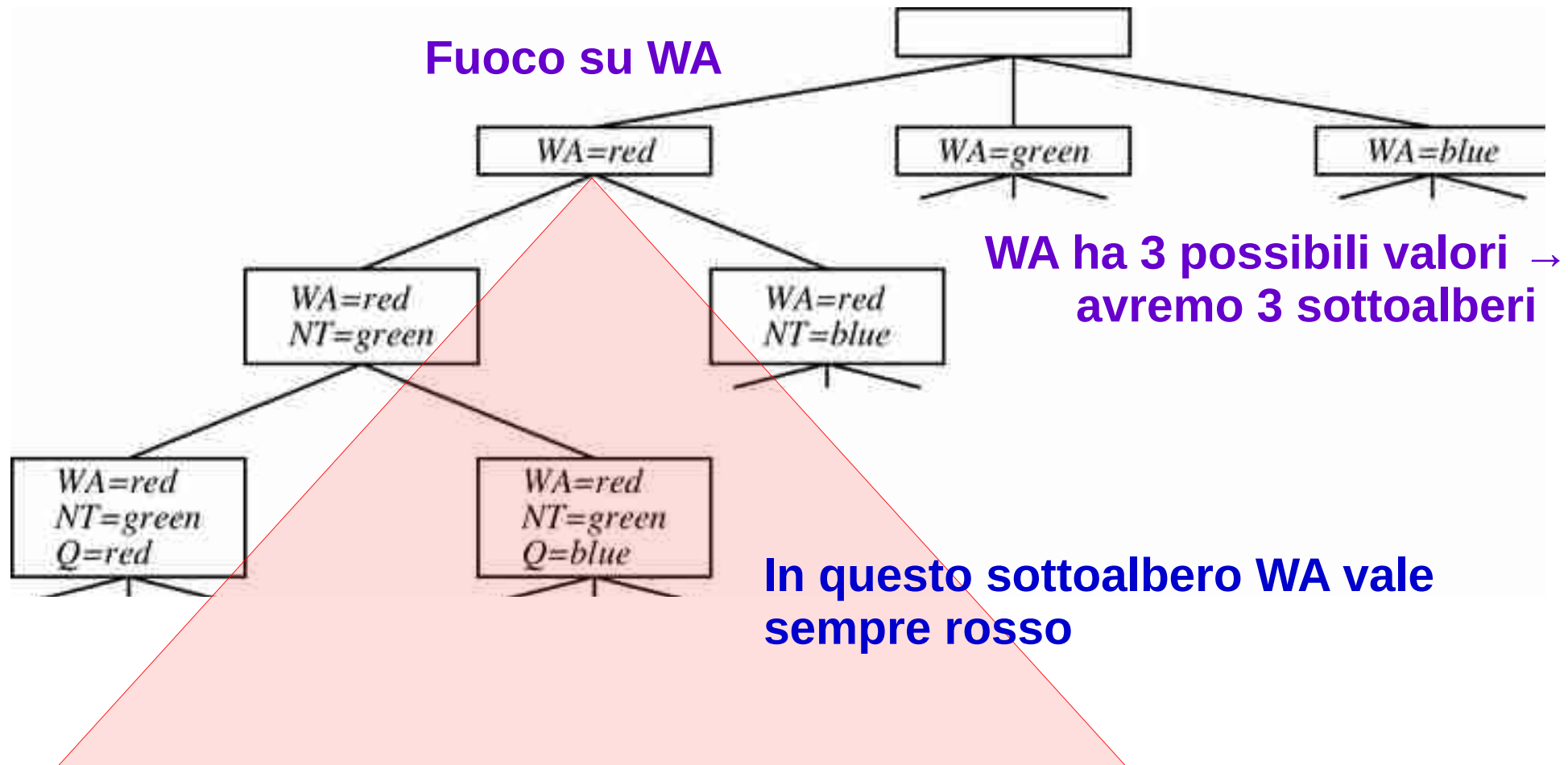


La stessa soluzione è ottenibile tramite più cammini, permutando l'ordine con cui le variabili vengono assegnate. La soluzione è sempre la stessa e l'ordine è ininfluenza sulla bontà della soluzione.

Gli algoritmi quindi **prima scelgono una variabile** e **poi un valore** per questa variabile

Esempio, Australia

Fuoco su WA



Con questa restrizione **il numero di foglie si riduce a d^n** , nel nostro esempio a **2.187** (un bel risparmio rispetto a 11.022.480)

Ricerca con backtracking: algoritmo

```
function BACKTRACKING-SEARCH(csp) returns a solution, or failure
  return RECURSIVE-BACKTRACKING({ }, csp)

function RECURSIVE-BACKTRACKING(assignment, csp) returns a solution, or failure
  if assignment is complete then return assignment
  var ← SELECT-UNASSIGNED-VARIABLE(VARIABLES[csp], assignment, csp)
  for each value in ORDER-DOMAIN-VALUES(var, assignment, csp) do
    if value is consistent with assignment according to CONSTRAINTS[csp] then
      add { var = value } to assignment
      result ← RECURSIVE-BACKTRACKING(assignment, csp)
      if result ≠ failure then return result
      remove { var = value } from assignment
  return failure
```

Figure 5.3 A simple backtracking algorithm for constraint satisfaction problems. The algorithm is modeled on the recursive depth-first search of Chapter 3. The functions SELECT-UNASSIGNED-VARIABLE and ORDER-DOMAIN-VALUES can be used to implement the general-purpose heuristics discussed in the text.

Questo algoritmo è riportato nella II edizione del libro ma non nella III

Ricerca con backtracking: algoritmo

```
function BACKTRACKING-SEARCH(csp) returns a solution, or failure
  return RECURSIVE-BACKTRACKING({ }, csp)

function RECURSIVE-BACKTRACKING(assignment, csp) returns failure
  if assignment is complete then return assignment
  var ← SELECT-UNASSIGNED-VARIABLE(VARIABLES[csp], assignment, csp)
  for each value in ORDER-DOMAIN-VALUES(var, assignment, csp) do
    if value is consistent with assignment according to CONSTRAINTS[csp] then
      add { var = value } to assignment
      result ← RECURSIVE-BACKTRACKING(assignment, csp)
      if result ≠ failure then return result
      remove { var = value } from assignment
  return failure
```

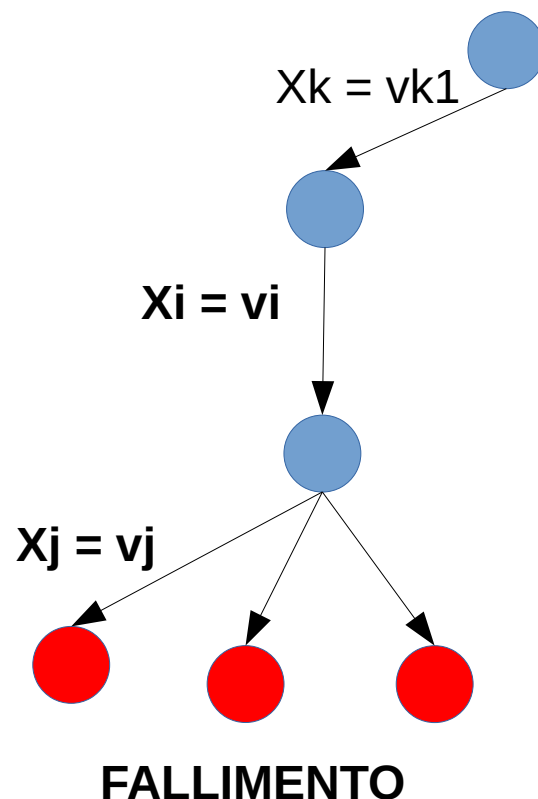
Figure 5.3 A algorithm is modeled on the recursive depth-first search of Chapter 3. The functions SELECT-UNASSIGNED-VARIABLE and ORDER-DOMAIN-VALUES can be used to implement the general-purpose heuristics discussed in the text.

Ricerca con backtracking: osservazioni

- È una ricerca in profondità, non informata
- Non è particolarmente efficiente, in particolare è soggetta a **thrashing** (lo stesso assegnamento errato può essere ripetuto in più punti dell'albero), esempio:

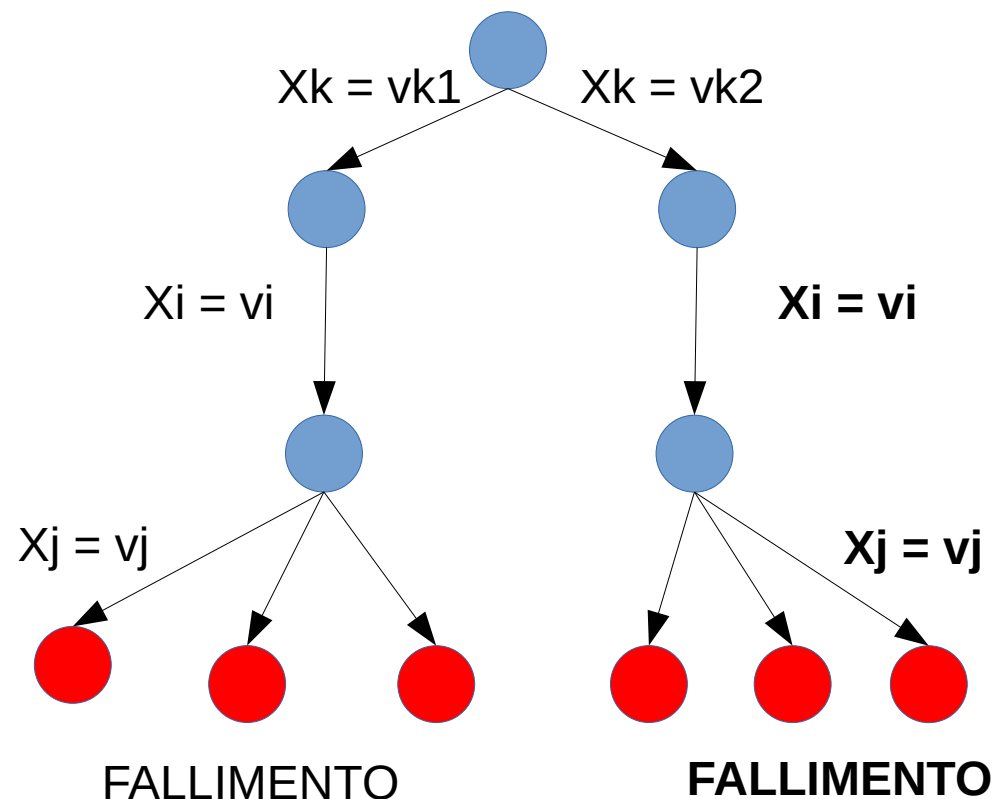
Ricerca con backtracking: osservazioni

- Supponiamo che il valore v_i di X_i sia incompatibile e quindi disabiliti tutti i valori di X_j . Questo sottoalbero è destinato a fallire



Ricerca con backtracking: osservazioni

- Se si fosse costretti a esplorare altri sottoalberi in cui nuovamente si considerano prima X_i e poi X_j si avrebbero altri fallimenti:



Ricerca con backtracking: osservazioni

- È una ricerca in profondità, non informata
- Non è particolarmente efficiente
- **Direzioni di miglioramento richiedono conoscenza sui vincoli**
 - 1) **L'assegnamento della variabile corrente** ha un impatto sulle variabili non ancora assegnate?
 - 2) **L'ordinamento dei valori** da provare influisce sul risultato?
 - 3) Quando un cammino fallisce, è possibile fare tesoro dell'esperienza per **evitare fallimenti simili** lungo altri cammini?

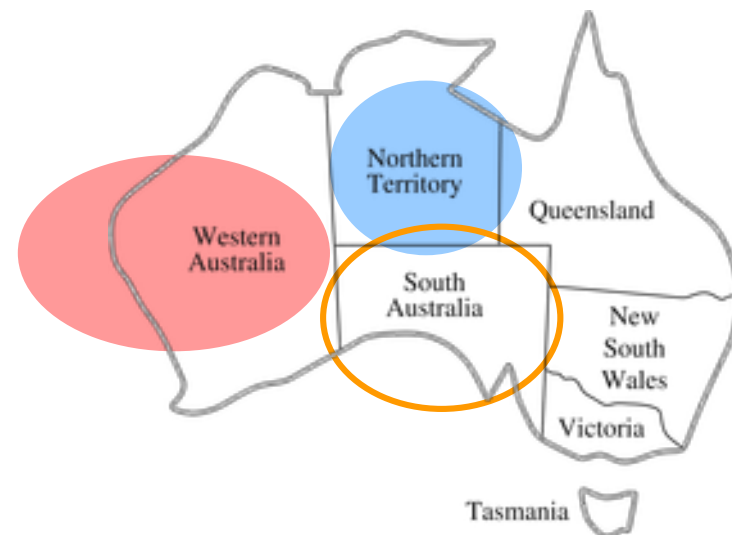
Euristiche di scelta della variabile

- L'algoritmo con backtracking sceglie, a ogni iterazione una variabile non assegnata:
 - *Finora abbiamo attuato scelte casuali*
 - *Se invece di una scelta casuale si adotta un'**euristica di scelta** si possono migliorare le prestazioni*
- **Esempio:** supponiamo si sia scelto il rosso per WA e il blu per NT, rimangono 5 variabili da assegnare
- Quale è più sensato scegliere?



Euristiche di scelta della variabile

- *La più vincolata*, cioè SA per la quale esiste uno solo valore possibile
- SA ha il vincolo di *non essere blu o rossa*, Q ha il vincolo di *non essere blu*, gli altri territori al momento potrebbero essere di qualsiasi colore
- **Euristica Minimum Remaining Values (o fail-first)**: sceglie una delle variabili con il minor numero di valori alternativi consistenti con l'assegnamento corrente



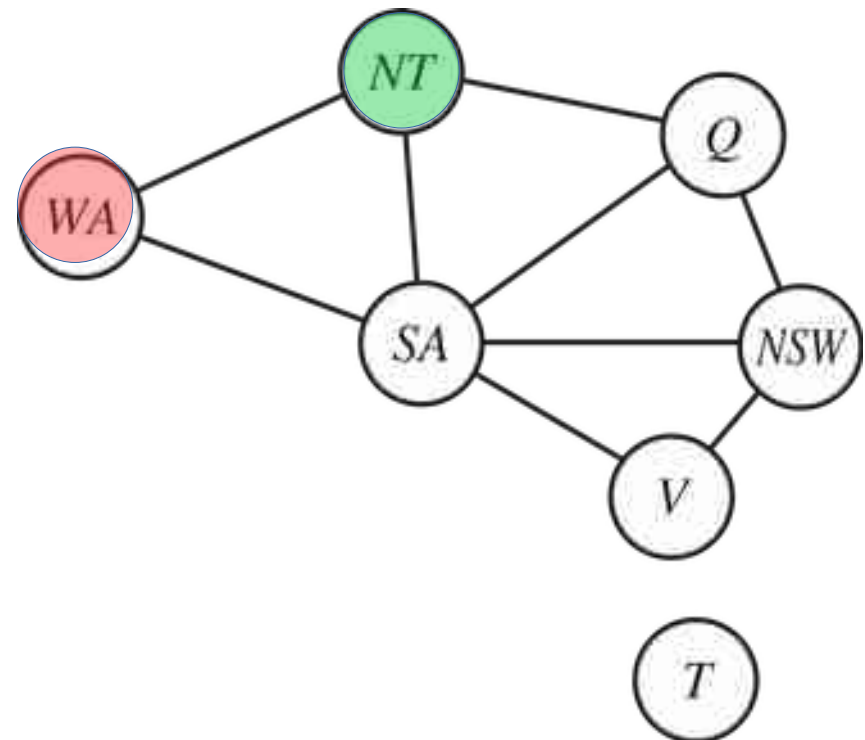
Euristiche di scelta della variabile

- *MRV non è sempre di aiuto (esempio, nella scelta della prima variabile da assegnare o in generale in caso di pareggio)*
- *In questo caso si usa l'**euristica di grado** che sceglie la variabile coinvolta in più vincoli*
- nell'esempio, se l'assegnamento iniziale fosse solo $NT=B$:
 - WA, SA e Q sarebbero a parimerito per MRV con due valori consistenti
 - l'euristica di grado sceglierebbe SA che è coinvolta in 5 vincoli

Euristiche di scelta del valore

- *Scelta una variabile che può assumere diversi valori, quale valore considerare prima?*
- **Euristica del valore meno vincolante:** prediligere il valore che lascia più libertà alle variabili adiacenti sul grafo dei vincoli

Supponiamo di avere assegnato **WA=R** e **NT=G** e di dover assegnare **Q**



Euristica del valore meno vincolante

Valori possibili per Q: **B** e **R**

Valori possibili per i vicini di Q:

SA: **B**

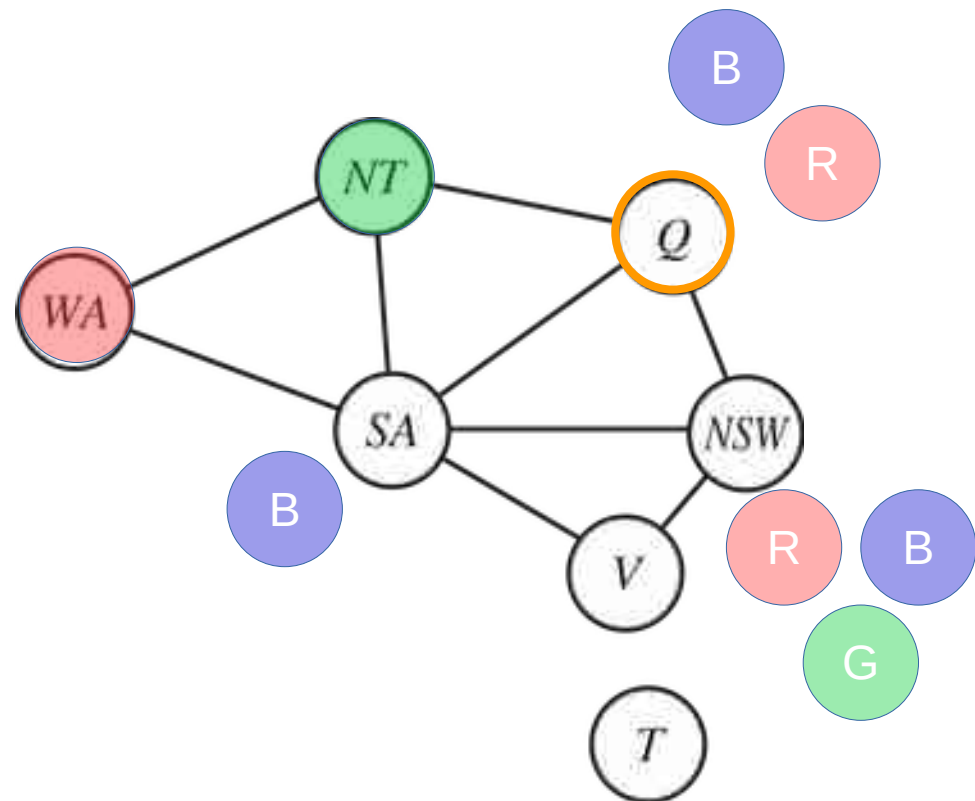
NSW: **R**, **G**, **B**

Se Q assume valore B: SA non può più assumere valori e i valori di NSW si riducono a 2

Se Q assume valore R: i valori possibili per SA non sono ridotti mentre NSW potrà essere G o B

È meno vincolante $Q=R$

Una scelta restrittiva anticipata rischia di tagliare via soluzioni



Rendiamo informata la ricerca

- La ricerca nello spazio dei possibili assegnamenti può essere ulteriormente migliorata usando meglio la conoscenza data dai vincoli
- Si alternano fasi di esplorazione a fasi di inferenza
- L'inferenza è espressa in termini di propagazione dei vincoli

Backtracking con inferenza

```
function RICERCA-BACKTRACKING(csp) returns una soluzione, o il fallimento
  return BACKTRACKING({ }, csp)

function BACKTRACKING(assegnamento, csp) returns una soluzione, o il fallimento
  if assegnamento è completo then return assegnamento
  var ← SCEGLI-VARIABILE-NON-ASSEGNATA(csp)
  for each valore in ORDINA-VALORI-DOMINIO(var, assegnamento, csp) do
    if valore è consistente con assegnamento then
      aggiungi {var = valore} ad assegnamento
      inferenze ← INFERENZA(csp, var, valore)
      if inferenze ≠ fallimento then
        aggiungi inferenze ad assegnamento
        risultato ← BACKTRACKING(assegnamento, csp)
        if risultato ≠ fallimento then
          return risultato
      rimuovi {var = valore} e inferenze da assegnamento
  return fallimento
```

Figura 6.5 Un semplice algoritmo di backtracking per problemi di soddisfacimento di vincoli. L'algo

Backtracking con inferenza

```
function RICERCA-BACKTRACKING(csp) returns una soluzione, o il fallimento
  return BACKTRACKING({ }, csp)

function BACKTRACKING(assegnamento, csp) returns una soluzione, o il fallimento
  if assegnamento è completo then return assegnamento
  var ← SCEGLI-VARIABILE-NON-ASSEGNATA(csp)
  for each valore in ORDINA-VALORI-DOMINIO(var, assegnamento, csp) do
    if valore è consistente con assegnamento then
      aggiungi {var = valore} ad assegnamento
      inferenze ← INFERENZA(csp, var, valore)
      if inferenze ≠ fallimento then
        aggiungi inferenze ad assegnamento
        risultato ← BACKTRACKING(assegnamento, csp)
        if risultato ≠ fallimento then
          return risultato
      rimuovi {var = valore} e inferenze da assegnamento
  return fallimento
```

EURISTICHE

Figura 6.5 Un semplice algoritmo di backtracking per problemi di soddisfacimento di vincoli. L'algoritmo

Consistenza locale

- **Scopo:** *ridurre lo spazio di ricerca in modo consistente usando i vincoli per tagliare assegnamenti non consistenti con l'informazione disponibile*
- **Metodo di consistenza locale:**
focalizzarsi su una **proprietà di consistenza relativa ad una parte del CSP** (un sottoproblema) e poi propagare questa informazione in tutto il CSP cercando di costruire una soluzione
- Fra le proprietà considerate:
 - 1) **Forward checking** (è un approccio più che una proprietà)
 - 2) **Node consistency:** riguarda singole variabili, vale quando i vincoli unari sono soddisfatti da tutti i valori dei domini delle rispettive variabili
 - 3) **Arc consistency:** proprietà direzionale relativa a un vincolo binario (vediamo più avanti), tutti i valori consistenti di una variabile x possono essere estesi a y tramite i vincoli
 - 4) **Path consistency:** proprietà che lega una coppia di variabili a una terza tramite i vincoli (vedremo più avanti)

Forward checking

- **Osservazione:** se $WA=R$ a causa dei vincoli le confinanti NT e SA non potranno assumere questo valore
- **Forward checking:**
si percorrono gli archi che collegano il nodo, corrispondente alla variabile assegnata, con i suoi vicini diretti e si riduce il range dei possibili valori di tali vicini in maniera conforme al vincolo
- **Esempio:**
quando WA diventa R si toglie R dai valori possibili di NT e SA
- Il forward checking si accompagna naturalmente con l'euristica MRV

Esempio

- La tabella riporta riduzioni progressive dei domini delle variabili. Vengono propagate dai vincoli attraverso al grafo dei vincoli a seguito degli assegnamenti provati. Quando una variabile (SA) non ha più valori legali si ha il fallimento e di conseguenza il backtracking

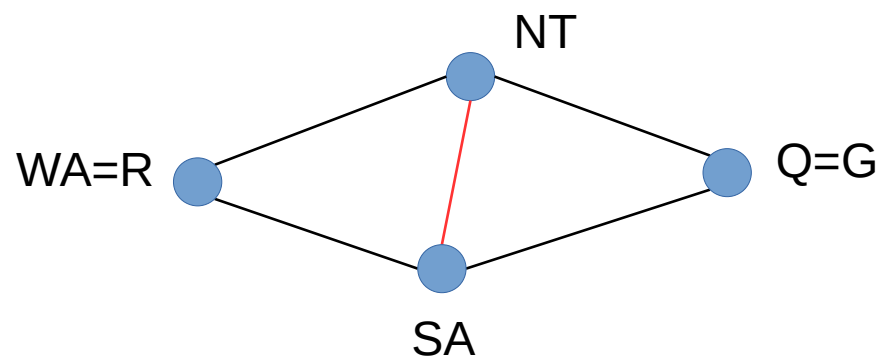
	WA	NT	Q	NSW	V	SA	T
INIZIO	R G B	R G B	R G B	R G B	R G B	R G B	R G B
WA=R →	R	G B	R G B	R G B	R G B	G B	R G B
Q=G →	R	B	G	R B	R G B	B	R G B
V=B →	R	B	G	R	B		R G B

Confinano tutti con SA

Limiti del forward checking

- Il forward checking non cattura tutte le inconsistenze

WA	NT	Q	NSW	V	SA	T
R G B	R G B	R G B	R G B	R G B	R G B	R G B
R	G B	R G B	R G B	R G B	G B	R G B
R	B	G	R B	R G B	B	R G B
R	B	G	R	B		R G B



Questo assegnamento forza un'inconsistenza infatti NT e SA, confinanti sono richiesti avere lo stesso valore!!

Forward checking non se ne accorge

Oltre il forward checking

- Node consistency
- Arc consistency
- Path consistency

Node consistency

- Un grafo di vincoli è **node consistent** quando lo sono tutte le sue variabili
- Una variabile è **node consistent** quando rispetta tutti i vincoli unari che la riguardano
- **Esempio:** “età compresa fra 18 e 35 anni” può essere espresso con due vincoli unari: $\text{età} > 18$ e $\text{età} < 35$
- L'algoritmo di node consistency controlla che un grafo di vincoli sia node consistent valutando tutte le variabili e i loro domini correnti rispetto ai vincoli unari

Arc consistency

- Un grafo di vincoli è **arc consistent** quando permette assegnamenti che rispettano tutti i vincoli binari
- È una proprietà importante perché: dato un vincolo a tre o più variabili è sempre possibile trasformarlo in un insieme di vincoli binari
- **NB:** il forward checking realizza un controllo di arc consistency per i vincoli che legano una variabile, alla quale è stato assegnato un valore, e i suoi vicini diretti sul grafo dei vincoli ma non propaga il ragionamento sui vicini dei vicini, ecc.

Arc consistency

- Dato un grafo di vincoli si intende per **arco** un *lato orientato*
- **Esempio:** $WA \rightarrow NSW$ sarà diverso da $NSW \rightarrow WA$
- Un arco è **consistente** quando:
per ogni valore possibile della variabile corrispondente al vertice sorgente, esiste un qualche valore del vertice destinazione che è ad esso consistente
- **Esempio:**
 - se il dominio di **WA** è $\{R, G\}$ e il dominio di **NSW** è $\{R\}$
 - **WA** $\rightarrow$ **NSW** non è consistente
se WA assume valore R, NSW non ha valori assegnabili
 - **NSW** $\rightarrow$ **WA** è consistente
- Si può cercare di rendere consistenti gli archi che non lo sono cancellando valori dai loro domini

- Algoritmo di arc consistency sviluppato nel 1977 da Alan Macworth[§] (AC-3 = Arc Consistency algorithm #3)
- Viene insegnato perché è più efficiente dei precedenti e più semplice dei successori
- Si può usare come **preprocessing** oppure a valle degli assegnamenti per **propagare le scelte** fatte tramite i vincoli. Quest'ultimo uso realizza l'algoritmo MAC (Maintaining Arc Consistency)

[§] se può essere utile sono accessibili delle slide introduttive ai CSP dello stesso Mackworth:
<http://www.cs.ubc.ca/~mack/CS322/lectures/3-CSP2.pdf>

function AC-3(*esp*) **returns** the CSP, possibly with reduced domains

inputs: *esp*, a binary CSP with variables $\{X_1, X_2, \dots, X_n\}$

local variables: *queue*, a queue of arcs, initially all the arcs in *esp*

while *queue* is not empty **do**

$(X_i, X_j) \leftarrow \text{REMOVE-FIRST}(\text{queue})$

if REMOVE-INCONSISTENT-VALUES(X_i, X_j) **then**

for each X_k **in** NEIGHBORS[X_i] **do**

 add (X_k, X_i) to *queue*

function REMOVE-INCONSISTENT-VALUES(X_i, X_j) **returns** true iff we remove a value

removed $\leftarrow$ false

for each x **in** DOMAIN[X_i] **do**

if no value y in DOMAIN[X_j] allows (x, y) to satisfy the constraint between X_i and X_j

then delete x from DOMAIN[X_i]; *removed* $\leftarrow$ true

return *removed*

Figure 5.7 The arc consistency algorithm AC-3. After applying AC-3, either every arc is arc-consistent, or some variable has an empty domain, indicating that the CSP cannot be made arc-consistent (and thus the CSP cannot be solved). The name “AC-3” was used by the algorithm’s inventor (Mackworth, 1977) because it’s the third version developed in the paper.